

EVIS: A Physics-Grounded Event Camera Plugin for NVIDIA Isaac Sim

Linli Shi Ruijun Zhang Ziyun Wang*

Johns Hopkins University

{lshi42, rzhan158}@jh.edu, claude.w@jhu.edu

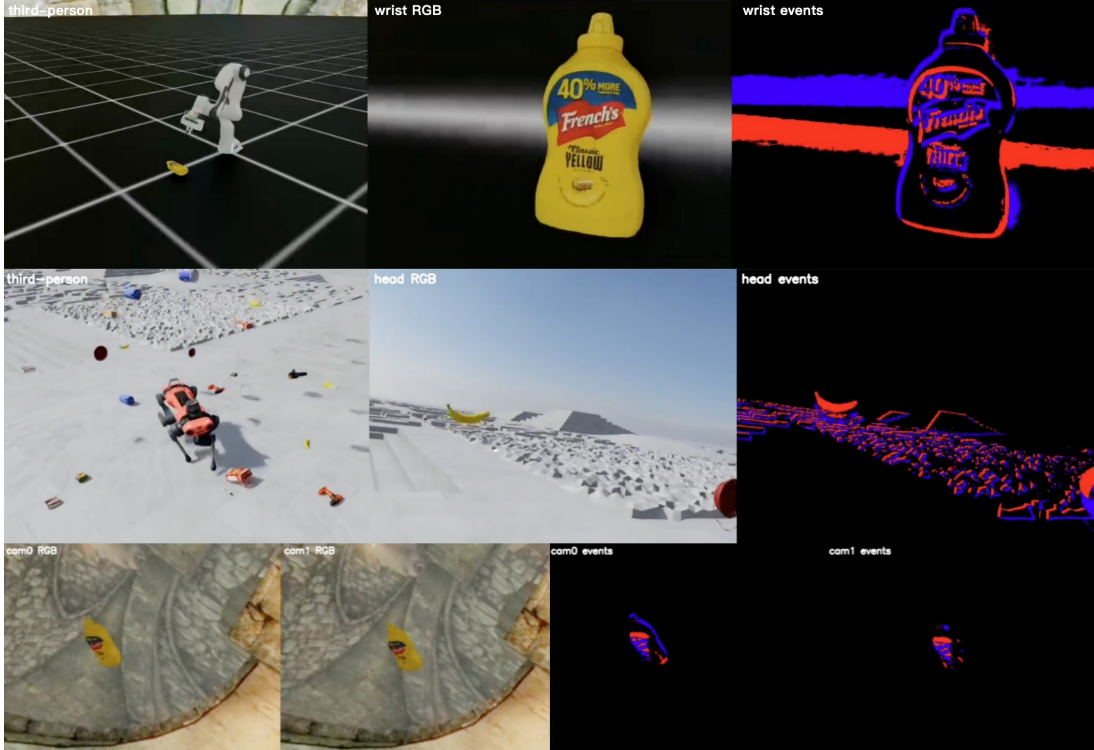


Figure 1. Top: a Franka wrist-mounted event camera circles a mustard bottle. Middle: a head-mounted event camera on a walking ANYmal quadruped as objects drop. Bottom: a stereo event camera observes a falling object. Videos in the [repository](#).

Abstract

Event cameras offer microsecond temporal resolution, low latency, and high dynamic range, making them attractive for robotics. However, labeled event-camera data for a specific robot and scene is scarce and expensive to collect, which slows the development of event-based perception and control. We present **EVIS: a physics-grounded event camera plugin for NVIDIA Isaac Sim** that generates high-rate, fully labeled event streams directly inside a physics simulator. The plugin implements a faithful log-intensity contrast event model with per-pixel asynchronous reference updates; it migrates from a normal RGB

camera with few changes and integrates into any Isaac Sim / Isaac Lab scene, inheriting the simulator’s physics and frame-perfect ground truth. It is fully configurable, and offers an interpolation option that renders only sparse keyframes and synthesizes the in-between frames through bidirectional motion-vector warping, making real-time generation on a single GPU possible. Optional sensor noise and motion blur further narrow the gap to real cameras. The generated streams are directly usable by pretrained event networks for downstream tasks. Code repository: <https://github.com/spikelab-jhu/isaac-sim-event-camera-plugin>.

1. Introduction

Event cameras do not output full synchronous frames like conventional cameras; instead they report per-pixel brightness changes asynchronously. This mechanism yields high temporal resolution, low latency, and high dynamic range, which makes them especially well-suited to fast-motion, contact-rich, or high-dynamic-range robotic settings. Yet the data ecosystem for event-based robotics remains thin. For a given robot embodiment and task, real event recordings are scarce, must be precisely labeled and aligned, and are costly to collect. Simulation is a natural remedy, but a good event-camera simulator must satisfy three requirements simultaneously.

(i) **Fidelity.** Events arise from brightness *changes*: the simulator needs a correct contrast-triggering model at a high effective frame rate, including the sensor’s noise processes.

(ii) **Physics consistency.** The events must come from a genuine physics-simulated scene with exact ground truth, not from a pre-recorded video.

(iii) **Rendering throughput.** Event generation must keep pace with the simulator, ideally at or above real time on a single GPU, so that event-based perception can be trained at the same scale as RGB pipelines.

To meet all three, we build the event-camera simulator directly inside NVIDIA Isaac Sim, integrated through the Isaac Lab framework. Our contributions are:

1. A faithful, GPU-batched **log-intensity contrast event model** with per-pixel asynchronous reference latching, validated on downstream tasks: pretrained event networks run unmodified on the generated streams.
2. A **direct integration with Isaac Sim**: replacing an RGB camera configuration with an event-camera configuration lets any physics-simulated scene produce labeled event data.
3. A **motion-vector frame-interpolation** pipeline that renders only sparse RTX keyframes and synthesizes the high-rate frames in between, enabling real-time operation on a single GPU.

2. Related Work

Event cameras have enabled a range of robotic perception tasks [5], including independent-motion segmentation [23], 3D reconstruction [21], wide-baseline feature matching [25], tracking [9], and high-speed object catching [22]. Progress on these tasks depends on labeled event data, yet real recordings for a specific robot and scene remain scarce and costly to collect and align. Large multi-sensor datasets such as M3ED [4] and recent effort OcToSense [3] provide valuable real-world event data, but they are necessarily tied to particular platforms, sensor layouts, and task domains; this motivates high-fidelity simulation for generating synchronized events, physics, and ground truth

across diverse robots and scenes.

Most event simulators convert an existing RGB video, possibly frame-interpolated, into events: v2e [10], Vid2E [6], PIX2NVS [2], DVS-Voltmeter [12], and V2CE [26], with further work on learned or domain-adaptive [8], differentiable [15], and text-conditioned [17] event models. These operate on pre-rendered frames and are decoupled from any physics simulator. A second line renders events directly from a scene or model rather than from a fixed video: ESIM [18] adaptively renders synthetic 3D scenes through a coupled rendering engine, the classic event-camera simulator [14] builds on Blender, EvD-NeRF [1] renders events from a dynamic neural radiance field, and a DVS plugin [11] adds event output to the Gazebo robot simulator. These are closest in spirit to us, but none is integrated with a modern GPU-parallel physics simulator. NVIDIA Isaac Sim and Isaac Lab [13] have become common platforms for GPU-parallel robot learning, and our work fills this gap: the event model runs directly inside Isaac Sim, so events, physics, and ground truth are produced together.

For downstream validation we use three representative tasks: E2VID [19] reconstructs intensity video from events, E-RAFT [7] estimates dense optical flow from consecutive event windows, and Match-Any-Events [25] matches events across two views; all run pretrained and frozen.

3. Event Model

By default, events are computed from the renderer’s HDR color buffer without gamma correction, which is linear in light intensity and therefore matches the response of real event-camera hardware. For each pixel, we first compute a luminance from the color values using Rec. 709 weights:

$$I = 0.2126 R + 0.7152 G + 0.0722 B. \quad (1)$$

We then take a log response, adding a small constant to avoid log 0:

$$L = \log(I + \epsilon), \quad \epsilon = 10^{-5}. \quad (2)$$

The plugin maintains a per-pixel reference log-intensity L_{ref} . When the difference between the current log-intensity and the reference exceeds a contrast threshold C , an event fires:

$$\Delta L = L - L_{\text{ref}}, \quad p = \begin{cases} +1 & \Delta L \geq +C \quad (\text{ON}) \\ -1 & \Delta L \leq -C \quad (\text{OFF}). \end{cases} \quad (3)$$

Whenever a pixel fires, its reference is updated to the current log-intensity, $L_{\text{ref}} \leftarrow L$. Thus every pixel independently performs a “fire-and-latch” process that mirrors the asynchronous operation of a real event camera. The entire update runs as batched tensor operations over all environments

and all cameras simultaneously. The default threshold is $C = 0.15$. Events are written to HDF5 as (x, y, t, p) tuples, grouped per camera.

4. Motion-Vector Frame Interpolation

RTX rendering dominates the pipeline’s cost, yet a faithful event stream needs a kHz-scale frame rate. Motivated by this, we provide a motion-vector based frame-interpolation option that allows users to render keyframes at a low rate f_k and synthesize the $K - 1$ frames inside each gap, giving the event model an effective rate of $f_k \cdot K$.

4.1. Bidirectional warping

At each keyframe we render the color frame, motion vectors, and the depth map. The renderer provides only one-directional motion vectors, mapping each pixel back to its position in the previous frame (earlier position = position + mv): the field $\mathbf{m}_B$ at keyframe B is thus exactly the $A \rightarrow B$ correspondence and directly drives the backward warp of B , while for A we approximately reuse its previous-step motion $\mathbf{m}_A$ as the forward velocity of its content. Linearizing the motion across the gap, we synthesize the intermediate frame at fraction $f = i/K$, $i \in \{1, \dots, K-1\}$, by forward-warping A , backward-warping B , and compositing the two.

Warping. A source pixel $\mathbf{p}$ of keyframe A is splatted *forward* to the intermediate time, and a pixel of B *backward*, at the target positions

$$\mathbf{q}_A(\mathbf{p}) = \mathbf{p} - f \mathbf{m}_A(\mathbf{p}), \quad \mathbf{q}_B(\mathbf{p}) = \mathbf{p} + (1-f) \mathbf{m}_B(\mathbf{p}), \quad (4)$$

where the minus sign reverses $\mathbf{m}_A$ (which points *backward* in time) into A ’s forward velocity. Since target positions are sub-pixel and several sources can land on the same target, each keyframe is resampled by softmax splatting [16]: for $X \in \{A, B\}$,

$$\hat{I}_X(\mathbf{q}) = \frac{\sum_{\mathbf{p}} b(\mathbf{q} - \mathbf{q}_X(\mathbf{p})) e^{\beta w_X(\mathbf{p})} I_X(\mathbf{p})}{\sum_{\mathbf{p}} b(\mathbf{q} - \mathbf{q}_X(\mathbf{p})) e^{\beta w_X(\mathbf{p})}}, \quad (5)$$

where $b(\mathbf{d}) = \max(0, 1 - |d_x|) \max(0, 1 - |d_y|)$ is the bilinear kernel and the importance $w_X(\mathbf{p})$ is the normalized inverse depth $1/(Z_X(\mathbf{p}) + \epsilon)$, so that when a foreground and a background pixel collide, the nearer surface wins ($\beta = 12$). Target pixels that receive no contribution (vanishing denominator) are marked as holes.

Compositing. The backward-warped B is the primary result, since $\mathbf{m}_B$ is the exact $A \rightarrow B$ correspondence. The forward-warped A only fills B ’s disocclusion holes, and pixels covered by neither stay black instead of fabricating

color:

$$I_f(\mathbf{q}) = \begin{cases} \hat{I}_B(\mathbf{q}) & \text{if } \mathbf{q} \text{ is covered by } B, \\ \hat{I}_A(\mathbf{q}) & \text{if } \mathbf{q} \text{ is covered by } A \text{ only,} \\ 0 & \text{otherwise.} \end{cases} \quad (6)$$

All intermediates across environments and cameras are batched into one scatter call per direction, which accelerates the warp stage.

4.2. Border handling

Pixels near the image border may lack valid neighbours after warping, producing artifacts. To address this, the plugin supports an optional over-render margin: a few extra pixels are rendered on each side and cropped off after warping. This yields event images without visible border artifacts.

4.3. Anti-aliasing

Isaac Lab enables anti-aliasing by default. This is harmless for full-resolution rendering, but it breaks the warp: anti-aliasing blends each object silhouette into a thin fringe of edge pixels that are alpha-composited with the background. These fringe pixels inherit the background’s near-zero motion vectors rather than those of the foreground object. As a result, under the warp they fail to track the object’s motion and are left behind as a faint ghosting artifact along the silhouette. We therefore disable anti-aliasing whenever interpolation is active, which eliminates the artifact; full-rendering paths remain unaffected.

5. Modeling Sensor Non-Idealities

The pipeline described so far is ideal: events are noise-free and frames have zero exposure time. We close this gap with two optional components, a sensor noise model and a motion blur model.

5.1. Sensor noise

Real event cameras deviate from the ideal model in well-documented ways [5, 10, 18]. We provide six optional noise components to model this, configurable per camera:

- **Threshold mismatch.** The scalar threshold C is replaced by per-pixel ON/OFF thresholds

$$\theta^{\text{ON}}, \theta^{\text{OFF}} \sim \max(\mathcal{N}(C, \sigma_\theta^2), \theta_{\min}), \quad (7)$$

sampled once per pixel, modeling the fixed-pattern mismatch of a real array.

- **Refractory period.** After a pixel fires at t_{last} , an event at time t is kept only if $t - t_{\text{last}} \geq \tau_r$.
- **Leak events.** Spurious ON events from junction leakage: in each synthesis step of duration Δt , a pixel leaks with probability

$$P_{\text{leak}} = r_\ell \Delta t. \quad (8)$$

- **Shot noise.** Random-polarity events whose rate grows with darkness:

$$P_{\text{shot}} = r_s \Delta t (1 - \bar{I}), \quad (9)$$

where $\bar{I} \in [0, 1]$ is the normalized intensity and the polarity is uniform.

- **Hot pixels.** A fixed random subset $\mathcal{H}$ fires at a high rate: $P_{\text{hot}} = r_h \Delta t$ for $\mathbf{p} \in \mathcal{H}$.
- **Finite bandwidth.** Darker pixels respond more slowly and lag the true signal. Following [10], we low-pass the log frame with an intensity-dependent time constant; the event model then operates on the filtered $\tilde{L}$ in place of L :

$$\tilde{L} \leftarrow (1 - \alpha) \tilde{L} + \alpha L, \quad \alpha = \min(\bar{I} \Delta t / \tau, 1), \quad (10)$$

with $\tau = 1/(2\pi f_c)$, giving an effective cutoff $f_c \bar{I}$ and a response time constant $\tau/\bar{I}$ that grows as a pixel darkens ($\bar{I}$ is floored at 0.02 so zero-light pixels do not freeze).

Sampling with probability $r \Delta t$ per step approximates a Poisson process with rate r , since $r \Delta t \ll 1$ at the kHz synthesis rate. All randomness comes from a single seeded generator, so noisy datasets are exactly reproducible.

The bottom two rows of Figure 5 visualize per-pixel threshold mismatch and leak events on the 1000 Hz point cloud. As shown, the per-pixel threshold mismatch introduces vertical striping roughness across the point cloud surface, while the leak events produce a sparse spatiotemporal noise haze throughout the cloud.

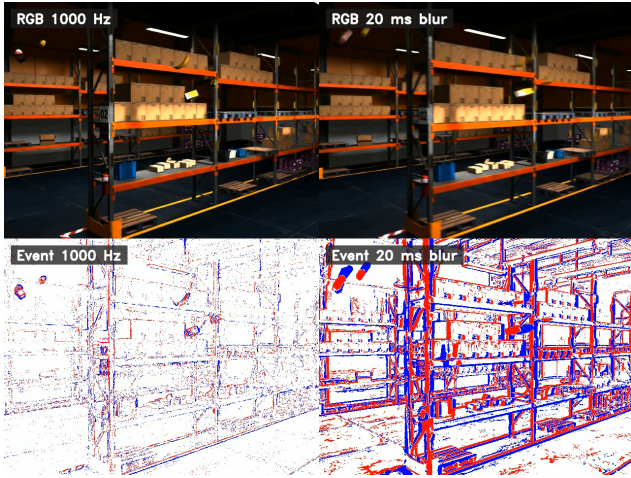


Figure 2. Motion blur with a 20 ms exposure. Left: sharp 1000 Hz frames and their 1 ms events. Right: the same instant with the exposure enabled; one batch of events covers the whole 20 ms of motion, smearing edges into wide bands.

5.2. Motion blur

A real frame camera integrates light over its shutter time, producing motion blur. We model this by averaging the fine

config	render	warp	event	total	vs. real-time
1000 (no warp)	13.18	—	0.70	13.88	13.9× slower
250 × 4	4.24	0.94	0.52	5.70	5.7× slower
250 × 4 + m50	4.32	1.06	0.58	5.96	6.0× slower
125 × 8	2.03	0.73	0.46	3.22	3.2× slower
125 × 8 + m50	2.06	1.07	0.44	3.57	3.6× slower
40 × 8	2.03	0.74	0.50	3.27	1.1× slower
40 × 8 + m50	1.99	1.08	0.46	3.53	1.1× slower
30 × 8	2.19	0.73	0.44	3.36	1.2× faster ✓
30 × 8 + m50	2.16	1.09	0.45	3.70	1.1× faster ✓

Table 1. Per-output-frame timing (ms) on a single RTX 5090, one environment. “config” is $f_{\text{render}} \times K$; “+m50” uses a 50-pixel over-render margin. Best, second-best, and third-best are highlighted in red, orange, and yellow.

frames already synthesized by the warp: for a shutter opening at t_0 with exposure E ,

$$B(\mathbf{q}) = \frac{1}{n} \sum_{j=0}^{n-1} I_{t_0+j\Delta t}(\mathbf{q}), \quad n = E/\Delta t, \quad (11)$$

where $\Delta t = 1/(f_k K)$ is the fine-frame step. Averaging the HDR frames approximates the radiance integral of a physical shutter. When motion blur is enabled, the event model operates on the blurred frames: one batch of events covers the entire motion within each exposure window, so edges are no longer thin contours but smear into wide bands. We show the motion blur result in Figure 2.

6. Evaluation

6.1. Real-time performance

Table 1 reports per-output-frame timing on a single RTX 5090, with one environment and one monocular 640×480 event camera. Rendering dominates the cost and decreases proportionally with the interpolation factor K , while the warp and event stages stay roughly constant. At moderate output rates the pipeline reaches real time: the 30×8 config generates 240 Hz events 1.2 times faster than real time. Throughput scales further with parallel environments, which amortize rendering across the batch.

6.2. Downstream-task test

A faithful event stream should be directly usable by event-based networks trained on *real* data. To show this, we run the publicly released pretrained models, frozen and *without any fine-tuning or adaptation* to our simulated events.

Test setup for reconstruction and optical flow estimation. The test scene uses the built-in Isaac Sim `warehouse_multiple_shelves` environment: the

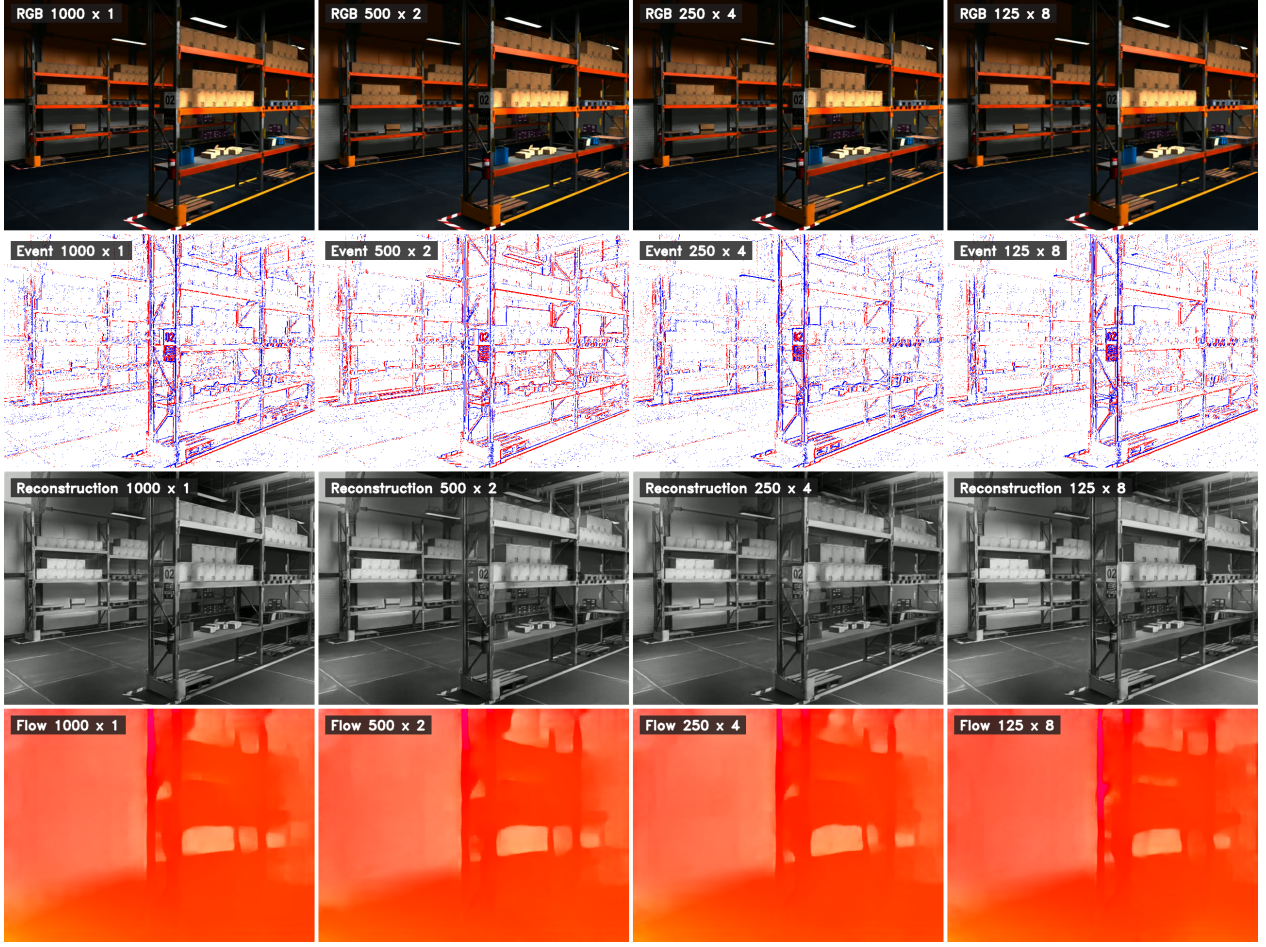


Figure 3. Qualitative overview of the downstream-task benchmark at one time instant. Columns: the full-render reference (1000 Hz) and the three warp configs (500×2, 250×4, 125×8), all at the same 1000 Hz effective rate. Rows, top to bottom: rendered frame, events (ON red, OFF blue), E2VID reconstruction, and E-RAFT optical flow.

camera views a multi-shelf aisle and sweeps a figure-8 trajectory (± 1.5 m horizontally, ± 0.8 m vertically, 0.5 Hz), producing egomotion events over the whole frame, while nine YCB objects are thrown through the view for independent object motion. Episodes are 4 s at 640×480 , with contrast threshold $C = 0.15$, the HDR event source, and noise disabled. We test four configs at the same 1000 Hz effective rate: a dense *full* reference rendered at 1000 Hz without warping, and three *warp* variants with sparser keyframes (500×2 , 250×4 , 125×8). Pretrained E2VID (reconstruction) and E-RAFT (optical flow) process every event stream, and their outputs are scored against the simulator’s ground truth sampled at 50 Hz. Figure 3 shows all four configs and all pipeline outputs at one matched instant.

Reconstruction. We reconstruct an intensity video from the events with the pretrained E2VID [19] network, windowed at the ground-truth timestamps (one 20 ms event

window per frame, matching the 50 Hz ground truth), and score each frame $\hat{I}$ against the ground-truth luminance I^* with three standard metrics: the mean squared error

$$\text{MSE} = \frac{1}{|\Omega|} \sum_{\mathbf{q} \in \Omega} (\hat{I}(\mathbf{q}) - I^*(\mathbf{q}))^2, \quad (12)$$

the structural similarity [20]

$$\text{SSIM} = \frac{(2\mu_{\hat{I}}\mu_{I^*} + c_1)(2\sigma_{\hat{I}I^*} + c_2)}{(\mu_{\hat{I}}^2 + \mu_{I^*}^2 + c_1)(\sigma_{\hat{I}}^2 + \sigma_{I^*}^2 + c_2)}, \quad (13)$$

computed over Gaussian windows and averaged, and LPIPS [24], the distance between the two frames in a pre-trained deep feature space. Since events encode only intensity *changes*, the absolute scale and offset of the reconstruction are unobservable; following the evaluation protocol of E2VID [19], we histogram-equalize both frames before scoring (CLAHE) so the metrics measure structure rather than global tone. Reconstruction error grows monotonically

as interpolation becomes more aggressive (Table 2): a gentle $2\times$ warp costs 0.047 SSIM relative to the full reference, while the aggressive $8\times$ warp costs 0.148.

config	interp.	SSIM $\uparrow$	MSE $\downarrow$	LPIPS $\downarrow$
1000	–	0.632	0.051	0.201
500×2	$2\times$	0.585	0.059	0.228
250×4	$4\times$	0.566	0.061	0.238
125×8	$8\times$	0.484	0.072	0.254

Table 2. Reconstruction result under interpolation on the warehouse scene (1000 Hz effective). E2VID reconstruction scored against the simulator’s ground-truth luminance (CLAHE-normalized, mean over frames). Best, second-best, and third-best are highlighted in red, orange, and yellow.

Optical flow. A unique benefit of generating events inside the simulator is exact dense ground-truth flow: the renderer’s per-pixel *motion vectors* give the true screen-space displacement between consecutive ground-truth timestamps, so no estimated or interpolated ground truth is needed. We estimate dense flow with the pretrained E-RAFT [7], feeding it two consecutive 20 ms event windows, and compare the predicted flow $\hat{\mathbf{f}}$ over the second window against the motion-vector ground truth $\mathbf{f}^*$ for the same 20 ms interval, over the event region, with the average endpoint error

$$\text{EPE} = \frac{1}{|\Omega|} \sum_{\mathbf{q} \in \Omega} \|\hat{\mathbf{f}}(\mathbf{q}) - \mathbf{f}^*(\mathbf{q})\|_2, \quad (14)$$

and the Pearson correlation between the predicted and ground-truth flow components. All configs are scored on the same windows. E-RAFT recovers the flow with subpixel accuracy from *every* config (Table 3); the bottom row of Figure 3 shows the recovered flow fields. This robustness to keyframe sparsity is expected by construction: the warp displaces content along the renderer’s own motion vectors, so the interpolated frames preserve the scene’s true motion field up to the constant-velocity assumption.

Feature matching. We match events across the two views with the pretrained Match-Any-Events matcher [25] over event-count windows of 150,000 events each, and score the matches against the simulator’s relative pose following the same protocol as [25]: matching precision, defined as the ratio of correct matches (epipolar distance below 1×10^{-4}) to total matches, and the AUC of the pose error at $5/10/20^\circ$, where the AUC is computed over five repeated RANSAC runs to reduce randomness. As on the other tasks, matching degrades gently with interpolation (Table 4). Figure 4 shows an example matched pair.

config	interp.	EPE (px) $\downarrow$	corr. $\uparrow$
1000	–	0.96	0.87
500×2	$2\times$	0.97	0.87
250×4	$4\times$	0.99	0.86
125×8	$8\times$	0.99	0.86

Table 3. Optical flow vs. ground truth on the warehouse scene: E-RAFT flow from each config’s events against the renderer’s motion-vector ground truth. Best, second-best, and third-best are highlighted in red, orange, and yellow.

config	Prec $\uparrow$	AUC@ 5° $\uparrow$	AUC@ 10° $\uparrow$	AUC@ 20° $\uparrow$
1000	81.2	45.4	62.3	73.2
500×2	80.8	45.5	59.9	70.2
250×4	77.3	36.2	50.4	65.5
125×8	74.2	32.2	43.9	61.1

Table 4. Event-to-event feature matching on the two-camera scene, all configs at the same 1000 Hz effective rate. Best, second-best, and third-best are highlighted in red, orange, and yellow.

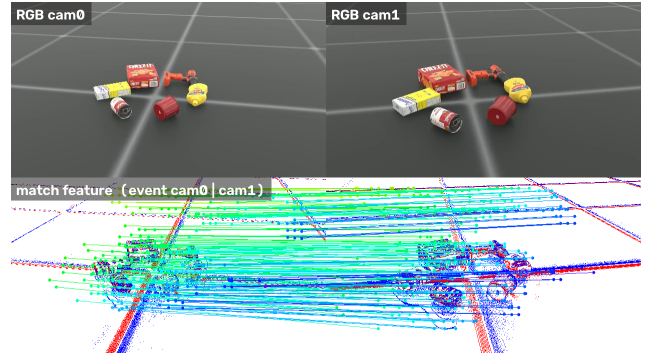


Figure 4. **Event-to-event feature matching.** Top: the RGB view from each of the two cameras, which converge on the same objects from a 20° separation. Bottom: the event streams of the two views (ON red, OFF blue) with the correspondences found by the pretrained matcher drawn as colored lines.

6.3. Qualitative result

Figure 1 showcases the plugin across several robot embodiments: a Franka wrist-mounted camera, a head-mounted camera on a walking ANYmal, and a stereo rig watching a free-falling object. In each case, the only change required was the camera configuration.

To visualize how the interpolation interacts with motion, we move a power drill in front of a *static* camera against a plain white background, so that every event comes from the drill, and record four canonical motions (free fall, out-of-plane roll, an approach toward the camera, and in-plane spin) under three temporal settings: our warp-accelerated 35×8 , a native 280 Hz render, and a native 1000 Hz render

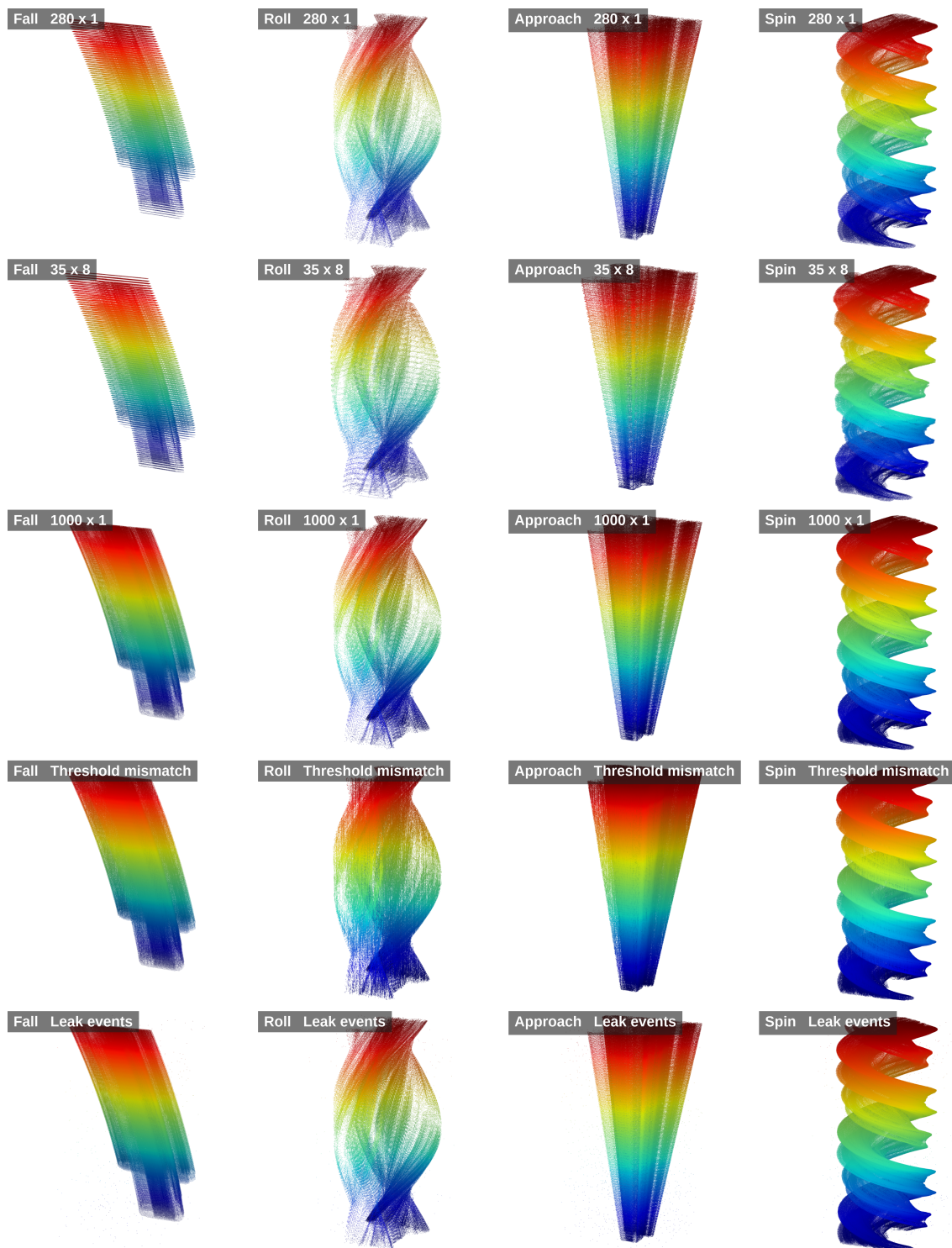


Figure 5. **Event point clouds across motions, temporal resolutions, and sensor noise.** Each point is a single event in space time (x, y, t) , colored by timestamp from blue (start of the clip) to red (end). A power drill undergoes four canonical motions (columns): free *fall*, out-of-plane *roll*, an *approach* toward the camera, and in-plane *spin*. The top three rows sweep temporal resolution: native 280 Hz, our warp-accelerated 35×8 , and native 1000 Hz. The warped stream reproduces the global spatiotemporal geometry of all four motions; under occlusion (roll), banding appears where surfaces revealed by rotation have no source pixels to warp from. The bottom two rows add sensor noise to the 1000 Hz stream: per-pixel *threshold mismatch* and *leak events*.

(Figure 5). Plotting every event as a point in (x, y, t) turns each stream into a spacetime surface. Across all four motions the warped 35×8 stream reproduces the same global geometry as the native renders, confirming that the interpolation preserves the true motion.

Two residual effects remain, both visible as a faint venetian-blind banding on the surface. The first is temporal quantization under fast displacement, seen in the fall: once the object moves several pixels per frame, consecutive time slices separate, and this appears even in the native 280 Hz render. The second is specific to the warp under rotation and occlusion, seen in the roll and spin. In the roll, occlusion means the surfaces revealed by rotation have no source pixels to warp from, so events pulse at the keyframe rate. In the spin, our assumption of linear motion in the warp produces slight deviations from the true rotation. Both effects are mild, and both shrink as the base render rate rises, as the cleaner 1000 Hz row shows. We therefore recommend raising the base render frequency when the target motion is fast or involves severe occlusion, trading part of the interpolation’s speedup for a smoother, more faithful event stream.

7. Conclusion

We presented EVIS: a physics-grounded event camera plugin for NVIDIA Isaac Sim that generates high-rate, fully labeled event streams directly inside a physics simulator. The plugin combines a GPU-batched log-intensity event model, a drop-in camera configuration that turns any Isaac Lab camera into an event camera, a motion-vector frame interpolation that reaches real-time generation on a single GPU, and optional sensor noise and motion blur. Every event is paired with the simulator’s exact ground truth, and pre-trained E2VID, E-RAFT, and Match-Any-Events run unmodified on the generated streams with reasonable performance. We hope this plugin lowers the barrier to event-based perception and control research and enables event-based pipelines to train at the same scale as their RGB counterparts.

References

- [1] Anish Bhattacharya, Ratnesh Madaan, Fernando Cladera, Sai Vemprala, Rogerio Bonatti, Kostas Daniilidis, Ashish Kapoor, Vijay Kumar, Nikolai Matni, and Jayesh K Gupta. Evdnerf: Reconstructing event data with dynamic neural radiance fields. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 5846–5855, 2024. 2
- [2] Yin Bi and Yiannis Andreopoulos. Pix2nvs: Parameterized conversion of pixel-domain video frames to neuromorphic vision streams. In *2017 IEEE International Conference on Image Processing (ICIP)*, pages 1990–1994. IEEE, 2017. 2
- [3] Anthony Bisulco, Jeremy Wang, Kostas Daniilidis, Randall Balestrero, and Pratik Chaudhari. Octosense: Self-supervised learning for multimodal robot perception. *arXiv preprint arXiv:2606.27317*, 2026. 2
- [4] Kenneth Chaney, Fernando Cladera, Ziyun Wang, Anthony Bisulco, M Ani Hsieh, Christopher Korpela, Vijay Kumar, Camillo J Taylor, and Kostas Daniilidis. M3ed: Multi-robot, multi-sensor, multi-environment event dataset. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4016–4023, 2023. 2
- [5] Guillermo Gallego, Tobi Delbrück, Garrick Orchard, Chiara Bartolozzi, Brian Taba, Andrea Censi, Stefan Leutenegger, Andrew J Davison, Jörg Conradt, Kostas Daniilidis, et al. Event-based vision: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 44(1):154–180, 2020. 2, 3
- [6] Daniel Gehrig, Mathias Gehrig, Javier Hidalgo-Carrió, and Davide Scaramuzza. Video to events: Recycling video datasets for event cameras. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 3586–3595, 2020. 2
- [7] Mathias Gehrig, Mario Millhäusler, Daniel Gehrig, and Davide Scaramuzza. E-raft: Dense optical flow from event cameras. In *2021 International Conference on 3D Vision (3DV)*, pages 197–206. IEEE, 2021. 2, 6
- [8] Daxin Gu, Jia Li, Yu Zhang, and Yonghong Tian. How to learn a domain-adaptive event simulator? In *Proceedings of the 29th ACM international conference on multimedia*, pages 1275–1283, 2021. 2
- [9] Friedhelm Hamann, Daniel Gehrig, Filbert Febryanto, Kostas Daniilidis, and Guillermo Gallego. Etap: Event-based tracking of any point. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 27186–27196, 2025. 2
- [10] Yuhuang Hu, Shih-Chii Liu, and Tobi Delbruck. v2e: From video frames to realistic dvs events. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1312–1321, 2021. 2, 3, 4
- [11] Jacques Kaiser, J Camilo Vasquez Tieck, Christian Hub-schneider, Peter Wolf, Michael Weber, Michael Hoff, Alexander Friedrich, Konrad Wojtasik, Arne Roennau, Ralf Kohlhaas, et al. Towards a framework for end-to-end control of a simulated vehicle with spiking neural networks. In *2016 IEEE International Conference on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAP)*, pages 127–134. IEEE, 2016. 2
- [12] Songnan Lin, Ye Ma, Zhenhua Guo, and Bihan Wen. Dvs-voltmeter: Stochastic process-based event simulator for dynamic vision sensors. In *European Conference on Computer Vision*, pages 578–593. Springer, 2022. 2
- [13] Mayank Mittal, Calvin Yu, Qinxu Yu, Jingzhou Liu, Nikita Rudin, David Hoeller, Jia Lin Yuan, Ritvik Singh, Yunrong Guo, Hammad Mazhar, et al. Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023. 2
- [14] Elias Mueggler, Henri Rebecq, Guillermo Gallego, Tobi Delbruck, and Davide Scaramuzza. The event-camera dataset and simulator: Event-based data for pose estimation, visual odometry, and slam. *The International journal of robotics research*, 36(2):142–149, 2017. 2

- [15] Jalees Nehvi, Vladislav Golyanik, Franziska Mueller, Hans-Peter Seidel, Mohamed Elgharib, and Christian Theobalt. Differentiable event stream simulator for non-rigid 3d tracking. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1302–1311, 2021. [2](#)
- [16] Simon Niklaus and Feng Liu. Softmax splatting for video frame interpolation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5437–5446, 2020. [3](#)
- [17] Joachim Ott, Zuowen Wang, and Shih-Chii Liu. Text-to-events: Synthetic event camera streams from conditional text input. In *2024 Neuro Inspired Computational Elements Conference (NICE)*, pages 1–10. IEEE, 2024. [2](#)
- [18] Henri Rebecq, Daniel Gehrig, and Davide Scaramuzza. Esim: an open event camera simulator. In *Conference on robot learning*, pages 969–982. PMLR, 2018. [2](#), [3](#)
- [19] Henri Rebecq, René Ranftl, Vladlen Koltun, and Davide Scaramuzza. High speed and high dynamic range video with an event camera. *IEEE transactions on pattern analysis and machine intelligence*, 43(6):1964–1980, 2019. [2](#), [5](#)
- [20] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, 13(4):600–612, 2004. [5](#)
- [21] Ziyun Wang, Kenneth Chaney, and Kostas Daniilidis. Evac3d: From event-based apparent contours to 3d models via continuous visual hulls. In *European conference on computer vision*, pages 284–299. Springer, 2022. [2](#)
- [22] Ziyun Wang, Fernando Cladera, Anthony Bisulco, Daewon Lee, Camillo J Taylor, Kostas Daniilidis, M Ani Hsieh, Daniel D Lee, and Volkan Isler. Ev-catcher: High-speed object catching using low-latency event-based neural networks. *IEEE Robotics and Automation Letters*, 7(4):8737–8744, 2022. [2](#)
- [23] Ziyun Wang, Jinyuan Guo, and Kostas Daniilidis. Un-evimo: Unsupervised event-based independent motion segmentation. In *European Conference on Computer Vision*, pages 228–245. Springer, 2024. [2](#)
- [24] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 586–595, 2018. [5](#)
- [25] Ruijun Zhang, Hang Su, Kostas Daniilidis, and Ziyun Wang. Match-any-events: Zero-shot motion-robust feature matching across wide baselines for event cameras. *arXiv preprint arXiv:2604.18744*, 2026. [2](#), [6](#)
- [26] Zhongyang Zhang, Shuyang Cui, Kaidong Chai, Haowen Yu, Subhasis Dasgupta, Upal Mahbub, and Tauhidur Rahman. V2ce: Video to continuous events simulator. In *2024 IEEE international conference on robotics and automation (ICRA)*, pages 12455–12461. IEEE, 2024. [2](#)